👩‍🏫 RAMÓN ABRAMO • GUÍA TÉCNICA DETALLADA

Versión 2: Teclado Virtual y Categorías

Evolución hacia una experiencia de usuario rica e interactiva con delegación de eventos, objetos semánticos y sincronización de teclado físico.

La **Versión 2** representa el salto de calidad más significativo en la usabilidad del juego. Sustituye el formulario estático por un teclado virtual generado dinámicamente con JavaScript, incorpora pistas temáticas por categorías y escucha en tiempo real las pulsaciones del teclado físico del usuario mediante eventos de ventana.

Estructura de la Carpeta `version-2-teclado-eventos/`

Archivo	Propósito	Destinatario
`index.html`	Interfaz con panel de categoría, botón de pista, casillas y contenedor del teclado.	Alumno / Docente
`main.css`	Estilos para teclado virtual, teclas con retroalimentación (verde/rojo) y corazones.	Alumno / Docente
`app.js`	Código maestro comentado detallando delegación de eventos y generación DOM.	Docente
`app.alumnos.js`	Plantilla con bloques `TODO` guiados para la práctica de aula.	Alumnado
`app.solucion.js`	Solución limpia de referencia para evaluación rápida del profesor.	Docente
`presentacion.html`	Diapositivas interactivas en pantalla completa para proyectar en clase.	Docente en Aula

Conceptos Avanzados de JavaScript Aplicados

Event Delegation

Un único escuchador en el contenedor `#teclado-virtual` gestiona los 27 botones mediante `e.target.closest('.tecla')`.

Estructura Set

Utilización de `new Set()` para garantizar comprobaciones de letras usadas en tiempo constante `O(1)`.

Sincronización Dual

El usuario puede jugar indistintamente haciendo clic en pantalla o presionando cualquier tecla de su ordenador.

[HITO 1] Banco de Datos con Objetos Semánticos

> En lugar de cadenas de texto aisladas, cada elemento del juego es ahora un objeto con información contextual rica:

```
const BANCO_PALABRAS = [
  {
    palabra: 'JAVASCRIPT',
    categoria: 'Programación Web',
    pista: 'Lenguaje interpretado interactivo que corre en el navegador.'
  },
  {
    palabra: 'CSS',
    categoria: 'Diseño Web',
    pista: 'Hojas de estilo en cascada para colores, maquetación y fuentes.'
  },
  {
    palabra: 'RESPONSIVE',
    categoria: 'Diseño Web',
    pista: 'Técnica para adaptar interfaces a móviles, tablets y monitores.'
  }
];

const LETRAS_ALFABETO = 'ABCDEFGHIJKLMNÑOPQRSTUVWXYZ'.split('');
const MAX_FALLOS = 6;
```

[HITO 2] La Colección Set para Letras Jugadas

En la Versión 1 usábamos un array tradicional y `includes()`. En la Versión 2 enseñamos la colección nativa `Set` de ES6:

```
// Declaración del conjunto reactivo
let letrasUsadas = new Set();

// Comprobación eficiente:
if (letrasUsadas.has(letra)) return;

// Inserción sin duplicados:
letrasUsadas.add(letra);

// Limpieza al reiniciar partida:
letrasUsadas.clear();
```

Valor Didáctico del Set frente al Array

Permite explicar la diferencia entre una colección secuencial indexada (Array) y un conjunto matemático de valores únicos (Set), introduciendo conceptos de eficiencia y legibilidad en el código.

[HITO 3] Inyección Dinámica del Teclado Virtual

> En lugar de escribir manualmente 27 etiquetas `<button>` en el archivo HTML, las generamos mediante programación JavaScript:

```
function renderizarTecladoVirtual() {
  tecladoVirtual.innerHTML = '';

  LETRAS_ALFABETO.forEach(letra => {
    const btn = document.createElement('button');
    btn.type = 'button';
    btn.className = 'tecla';
    btn.textContent = letra;
    btn.dataset.letra = letra; // data-letra="A"
    btn.setAttribute('aria-label', `Letra ${letra}`);
    tecladoVirtual.appendChild(btn);
  });
}
```

Renderizado de Casillas de Letras Individuales

Para elevar la estética visual, cada letra oculta se renderiza como una casilla independiente con bordes y animaciones CSS:

```
function actualizarPalabraVisual() {
  palabraDisplay.innerHTML = '';

  for (let i = 0; i < partidaActual.palabra.length; i++) {
    const casilla = document.createElement('span');
    casilla.className = 'casilla-letra';

    const letraDescubierta = letrasAdivinadas[i];
    if (letraDescubierta) {
      casilla.textContent = letraDescubierta;
      casilla.classList.add('revelada'); // Activa animación CSS
    }

    palabraDisplay.appendChild(casilla);
  }
}
```

> **Los atributos dataset en el DOM**
>
> El uso de `btn.dataset.letra = letra` inyecta en el HTML el atributo `data-letra="..."`. Esto desacopla el texto visible del botón del dato que procesa la lógica en JavaScript.

[HITO 4] Delegación de Eventos (Event Bubbling)

> En vez de asociar 27 escuchadores de eventos con `forEach`, vinculamos **un único escuchador** en el contenedor padre:

```
// Delegación de eventos óptima
tecladoVirtual.addEventListener('click', evento => {
  // Busca el botón con clase .tecla más cercano al clic
  const boton = evento.target.closest('.tecla');

  // Si se hizo clic fuera o la tecla ya está desactivada, ignoramos
  if (!boton || boton.disabled) return;

  // Obtenemos la letra del atributo dataset
  const letra = boton.dataset.letra;
  jugarLetra(letra);
});
```

Soporte Universal de Teclado Físico (keydown)

El usuario puede escribir con el teclado físico de su ordenador y el juego responde exactamente igual que si pulsara en pantalla:

```
window.addEventListener('keydown', evento => {
  if (juegoTerminado) return;

  // Ignoramos combinaciones de sistema (Ctrl+C, Alt+Tab, etc.)
  if (evento.ctrlKey || evento.altKey || evento.metaKey) return;

  const letraPulsada = evento.key.toUpperCase();

  // Si la tecla pulsada pertenece al alfabeto válido, la jugamos
  if (LETRAS_ALFABETO.includes(letraPulsada)) {
    jugarLetra(letraPulsada);
  }
});
```

Ventaja de closest()

Garantiza que el evento se capture correctamente aunque el botón contenga iconos internos o etiquetas anidadas como `<span>`.

Eficiencia en Memoria

1 solo listener en memoria consume una fracción de los recursos que consumirían 27 listeners individuales.

[HITO 5] Función Unificada `jugarLetra(letra)`

> Centraliza la ejecución tanto para clics virtuales como para pulsaciones de teclado físico:

```
function jugarLetra(letra) {
  if (juegoTerminado || letrasUsadas.has(letra)) return;
  letrasUsadas.add(letra);

  // Localizamos la tecla en pantalla y la desactivamos
  const boton = tecladoVirtual.querySelector(`[data-letra="${letra}"]`);
  if (boton) boton.disabled = true;

  if (partidaActual.palabra.includes(letra)) {
    if (boton) boton.classList.add('acierto'); // Estilo verde
    for (let i = 0; i < partidaActual.palabra.length; i++) {
      if (partidaActual.palabra[i] === letra) letrasAdivinadas[i] = letra;
    }
  } else {
    if (boton) boton.classList.add('fallo'); // Estilo rojo tachado
    fallos++;
    if (partesAhorcado[fallos - 1]) {
      partesAhorcado[fallos - 1].classList.add('visible');
    }
  }

  actualizarPalabraVisual();
  actualizarMarcadorVidas();
  verificarFinJuego();
}
```

[HITO 6] Pistas y Marcador con Corazones

Sistema de Pistas

Al pulsar "💡 Pedir Pista", se muestra el texto descriptivo del objeto y el botón se desactiva para evitar abusos.

Feedback con Corazones

Se recorren los 6 corazones: los que superen las vidas restantes adquieren la clase `.perdido` pasando a escala de grises.

Solución Canónica Completa (app.solucion.js)

```
const BANCO_PALABRAS = [
  { palabra: 'JAVASCRIPT', categoria: 'Web', pista: 'Lenguaje interactivo.' },
  { palabra: 'RESPONSIVE', categoria: 'Diseño', pista: 'Adaptabilidad a pantallas.' }
];
const LETRAS_ALFABETO = 'ABCDEFGHIJKLMNÑOPQRSTUVWXYZ'.split('');
const MAX_FALLOS = 6;
let partidaActual = null, letrasAdivinadas = [], letrasUsadas = new Set(), fallos = 0, juegoTerminado = false;

function renderizarTecladoVirtual() {
  tecladoVirtual.innerHTML = '';
  LETRAS_ALFABETO.forEach(letra => {
    const btn = document.createElement('button');
    btn.className = 'tecla'; btn.textContent = letra; btn.dataset.letra = letra;
    tecladoVirtual.appendChild(btn);
  });
}

function jugarLetra(letra) {
  if (juegoTerminado || !LETRAS_ALFABETO.includes(letra) || letrasUsadas.has(letra)) return;
  letrasUsadas.add(letra);
  const boton = tecladoVirtual.querySelector(`[data-letra="${letra}"]`);
  if (boton) boton.disabled = true;
  if (partidaActual.palabra.includes(letra)) {
    if (boton) boton.classList.add('acierto');
    for (let i = 0; i < partidaActual.palabra.length; i++)
      if (partidaActual.palabra[i] === letra) letrasAdivinadas[i] = letra;
  } else {
    if (boton) boton.classList.add('fallo');
    fallos++;
    if (partesAhorcado[fallos - 1]) partesAhorcado[fallos - 1].classList.add('visible');
  }
  actualizarPalabraVisual(); actualizarMarcadorVidas(); verificarFinJuego();
}

tecladoVirtual.addEventListener('click', e => {
  const btn = e.target.closest('.tecla');
  if (btn && !btn.disabled) jugarLetra(btn.dataset.letra);
});

window.addEventListener('keydown', e => {
  if (e.ctrlKey || e.altKey) return;
  const l = e.key.toUpperCase();
  if (LETRAS_ALFABETO.includes(l)) jugarLetra(l);
});
```

¡Enhorabuena! Has completado el análisis de la Versión 2 con Teclado Virtual. ⌨️ 💜